

The Bucket List DXB

Owner's Handbook

How to run, change and grow your website
working together with an AI assistant.

Written for the new owners.
No programming experience assumed.

What is in this handbook

Work through Part 1 once, on your own computer. After that, dip into whichever part you need. Nothing here assumes you can write code.

Part	What it covers	When to read it
1	Setting up your computer	Once, at the start
2	Talking to Claude Code	Before your first change
3	Everyday tasks, step by step	Whenever you want a change
4	Publishing changes to the live site	Every time you change something
5	Running the business day to day	Ongoing
6	Rules that keep you safe	Read once, remember forever
7	When something goes wrong	As needed
8	Plain-English dictionary	Whenever a word confuses you

First, what you actually own

Your product is **three separate websites** that all share **one database**. That last part matters: a change to the database affects all three.

Website	Address	Who uses it
The public site	tbdxb.vercel.app	Your customers. English and Arabic
Vendor dashboard	vendor-tbdxb.vercel.app	Businesses that list experiences with you
Admin dashboard	admin-tbdxb.vercel.app	You and your staff

Behind them sit four services. You will need an account for each.

Service	What it does for you	What happens if it breaks
Supabase	Stores all data: events, bookings, customer accounts, photos	Everything stops. This is the most important one
Vercel	Runs the three websites	The sites go offline, but no data is lost
GitHub	Stores the code and its full history	Sites keep running; you cannot publish changes
Resend	Sends the sign-in code emails	Customers cannot sign in

Part 1 - Setting up your computer

You only do this once. Set aside about an hour. If a step fails, skip to Part 7 at the back.

1 Get access to the four services

Before anything else, make sure you can sign in to all four. The previous owner transfers these to you.

- **Supabase** - supabase.com. You should see a project named **tbdxb**.
- **Vercel** - vercel.com. You should see **three** projects.
- **GitHub** - github.com. You should see the code repository.
- **Resend** - resend.com. Used for sending emails.

STOP - Do not skip this

If you cannot sign in to all four, stop and sort that out first. Everything below depends on it.

2 Install the free tools

Three things to install. Take them in order.

a) Node.js - the engine that runs the websites

Go to nodejs.org and download the version marked **LTS**. Open the file and click through the installer.

b) Git - keeps the history of every change

Go to git-scm.com/downloads, download for your system, and install it. On a Mac it may already be installed.

c) Claude Code - your AI developer

Go to claude.ai/code and follow the install instructions. You will need a Claude account, which is a paid subscription.

3 Open the Terminal

The Terminal is a window where you type commands instead of clicking. It looks intimidating and is not. You will use about six commands, ever.

- **Mac:** press Cmd+Space, type **Terminal**, press Enter.
- **Windows:** press the Start button, type **Terminal**, press Enter.

Check the tools installed correctly by typing this and pressing Enter:

TYPE THIS INTO THE TERMINAL

```
node --version
```

You should see something like **v22.1.0**. Any number is fine. If you instead see *command not found*, Node.js did not install - try Step 2a again.

4 Download your code

In GitHub, open your repository and click the green **Code** button, then copy the HTTPS address. It ends in **.git**.

In the Terminal, type this, replacing the address with yours:

TYPE THIS INTO THE TERMINAL

```
cd ~/Documents
git clone https://github.com/YOUR-NAME/tbdx.git
cd tbdx
```

You now have a copy of the whole project in **Documents/tbdx** on your computer.

What just happened

cd means change directory - it walks you into a folder. **git clone** downloads the project. You will use **cd** often; the others rarely.

5 Add the secret keys

The code needs passwords to reach your database. These are kept in files that are deliberately never uploaded to GitHub, so you create them by hand once.

Open the **tbdx** folder in Finder or File Explorer. You will create **three** files, all named **.env.local** - note the dot at the front:

- One in the main **tbdx** folder
- One in **tbdx/apps/vendor**
- One in **tbdx/apps/admin**

Easiest way: ask Claude Code to do it for you. Start it first (Step 6), then use this:

TYPE THIS INTO CLAUDE CODE

Create the three **.env.local** files I need for local development, using **.env.local.example** as the template. Leave the values blank and tell me exactly which value goes where.

Then fill in the values from **Supabase, Settings, API Keys**:

Value in the file	Where to find it in Supabase
NEXT_PUBLIC_SUPABASE_URL	Settings, Data API - the Project URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Settings, API Keys - the publishable key (starts sb_publishable_)
SUPABASE_SECRET_KEY	Settings, API Keys - the secret key (starts sb_secret_). Only needed in the vendor and admin files

STOP - Always use the copy button

Never select a key with your mouse and copy it. The key wraps onto two lines on screen, and dragging across the wrap silently adds a space in the middle. That breaks it, and the error message you get will not tell you why. This has already happened once on this project. Use the copy icon next to the key, every time.

6 Start Claude Code and say hello

In the Terminal, make sure you are in the project folder, then start it:

TYPE THIS INTO THE TERMINAL

```
cd ~/Documents/tbidxb
claude
```

Claude Code starts and reads the project's built-in instructions automatically. Your very first message should be this:

TYPE THIS INTO CLAUDE CODE

**I am the new owner of this project and I am not a developer.
Read CLAUDE.md and docs/README.md, then explain in plain English:
what this project is, what the three apps do, and what I should
be careful about.**

Read the answer. This is the moment where the project stops being a mystery.

7 Run the sites on your own computer

Before changing anything, get the site running locally. This is your practice space - nothing you do here touches the live site or real customers.

TYPE THIS INTO CLAUDE CODE

Start the public site on my computer so I can see it in a browser.

Claude Code will run it and give you an address, usually **http://localhost:3000**. Open that in your browser. That is your website, running on your own machine.

The other two work the same way:

Site	Local address
Public site	http://localhost:3000
Vendor dashboard	http://localhost:3200
Admin dashboard	http://localhost:3300

You are set up

From here on, the loop is always the same: describe what you want, look at it locally, then publish. Part 3 walks through it.

Part 2 - Talking to Claude Code

Claude Code is a developer that works by conversation. It can read every file, write code, run the site, and publish changes. The quality of what you get back depends almost entirely on how you ask.

The five habits that matter

1. Say what you want, not how to build it

You are the client, not the engineer. Describe the outcome.

Weaker	Better
Change the CSS flex property on the booking panel	The Book Now button is hard to see on mobile. Make it stand out more
Add a column to the event table	Vendors need to say whether an experience is wheelchair accessible

2. One change at a time

Ask for a single thing, check it, then ask for the next. A list of eight changes in one message gets you eight half-checked changes, and when something breaks you will not know which one did it.

3. Always ask it to check its own work

This is the single most valuable habit in this handbook. Add this sentence to the end of any request that changes something:

TYPE THIS INTO CLAUDE CODE

After making the change, run the build to make sure nothing is broken, then show me the result in the browser.

4. Ask it to explain before it does anything big

If a request feels large or risky, ask for the plan first:

TYPE THIS INTO CLAUDE CODE

Do not change anything yet. Explain what you would change, which files, and what could go wrong. Then wait for me.

5. Say when you do not understand

It will happily re-explain at any level. There is no penalty for asking, and a confused owner approving changes is the real risk.

TYPE THIS INTO CLAUDE CODE

Explain that again as if I have never seen code before.

Prompts worth keeping

Copy these. They work as written.

Starting any working session

TYPE THIS INTO CLAUDE CODE

Give me a short summary of the current state of the project and anything that needs my attention.

Understanding something before you change it

TYPE THIS INTO CLAUDE CODE

Explain how [the booking process] works, in plain English, step by step. Do not change anything.

Checking whether an idea is sensible

TYPE THIS INTO CLAUDE CODE

I am thinking of [adding a gift voucher feature].
Is that a small change or a big one for this project?
What would it touch, and what would you recommend?

Ending a session safely

TYPE THIS INTO CLAUDE CODE

Is there anything uncommitted or half-finished right now?
If yes, tell me what and whether it is safe to leave it.

A note on trust

Claude Code will tell you when it is unsure, and will say when it has not verified something. Read those caveats - they are the difference between 'it is done' and 'it should work'. If it says it could not test something, test it yourself before publishing.

Part 3 - Everyday tasks, step by step

Every task below follows the same four beats: **ask**, **look**, **approve**, **publish**. Once you have done one, you have done them all.

Task A - Change wording on the public site

The easiest kind of change, and a good first one to practise on.

- 1 Start Claude Code in the project folder.
- 2 Describe the change and where it is.
- 3 Look at it in your browser.
- 4 Publish it.

TYPE THIS INTO CLAUDE CODE

On the home page, the heading says
'Things worth doing in Dubai'.
Change it to 'Unforgettable experiences in Dubai'.
Remember this site is bilingual, so update Arabic too.
Then show me the result.

STOP - Never leave Arabic behind

The public site is fully bilingual. Every piece of text exists twice. Claude Code knows this, but say it anyway - an English phrase appearing in the Arabic site looks careless to customers.

Task B - Change how something looks

TYPE THIS INTO CLAUDE CODE

The category headings on the home page feel too small.
Make them larger and easier to read, but keep the existing
style of the site. Show me before and after.

For anything visual, ask to see it rather than trusting a description. Add: *show me a screenshot at phone size and at desktop size*.

Task C - Add a whole new page

TYPE THIS INTO CLAUDE CODE

Add a new page to the public site at /gift-cards.
It should match the style of the existing About Us page.
Content: a heading, two paragraphs I will write later,
and a contact button. Add it to the footer links.
English and Arabic. Then show it to me.

Then fill in the words:

TYPE THIS INTO CLAUDE CODE

On the gift cards page, replace the placeholder text with this:
[paste your text here]

Task D - Fix something that is broken

Describe the symptom precisely: what you did, what you expected, what happened instead. A screenshot helps enormously - you can drag an image straight into the chat.

TYPE THIS INTO CLAUDE CODE

On the vendor dashboard, when I click Save on the event form, nothing appears to happen. I expected a confirmation. Investigate and tell me what is wrong before fixing it.

Task E - Store a new piece of information

This is the one genuinely risky category, because it changes the **database** - the thing that holds all your real customer and booking data. It needs one extra manual step that nothing else does.

STOP - Read this whole section before starting

Database changes are applied by hand and are not automatically reversible. Do them calmly, one at a time, and never late at night before a busy weekend.

Say what you want to store, in business terms:

TYPE THIS INTO CLAUDE CODE

Vendors need to mark whether an experience is wheelchair accessible, and customers should see that on the event page.
Explain the plan first, including the database change.
Do not write anything yet.

Once you are happy with the plan, let it proceed. It will write a **migration** - a small file of database instructions. Then:

- 1 Claude Code gives you the migration file and tells you it is ready.
- 2 Open **Supabase**, then **SQL Editor**, then **New query**.
- 3 Paste the contents of the migration file in.
- 4 Press **Run**.
- 5 You should see *Success. No rows returned*. Tell Claude Code it worked.
- 6 Only now publish the code, following Part 4.

STOP - Order matters

Always run the database change BEFORE publishing the code that uses it. The other way round, your live site will break for however long the gap is.

Ask for the safety check

Add this to any database request: *explain in plain English what this migration does, and confirm it cannot delete existing data.*

Task F - Something you have not thought of

The pattern generalises. Describe the outcome, ask for a plan, approve, look, publish.

TYPE THIS INTO CLAUDE CODE

I want to [describe your idea in your own words].

Explain how you would do it and what it affects.

Do not change anything yet.

Part 4 - Publishing changes to the live site

Changes you make on your computer are invisible to the world until you publish them. Publishing is one instruction.

The safe publishing routine

1 Check it locally first

TYPE THIS INTO CLAUDE CODE

Show me the change in the browser so I can check it before we publish.

2 Ask for the publish

TYPE THIS INTO CLAUDE CODE

This looks good. Commit the change with a clear message and push it to main.

Commit means save a labelled snapshot. **Push to main** means upload it, which triggers the live update automatically.

3 Wait, then check the real site

Vercel rebuilds automatically, usually in one to three minutes. Then open the real address and confirm with your own eyes.

TYPE THIS INTO CLAUDE CODE

Confirm the change is live and everything still loads.

If you publish something you regret

Every change is a labelled snapshot, so going back is quick. Do not panic and do not start deleting things.

TYPE THIS INTO CLAUDE CODE

The last change I published caused a problem. Undo it and put the site back to how it was before, then explain what went wrong.

STOP - The one thing that does not simply undo

Code changes reverse easily. **Database** changes may not - if a migration deleted a column, the data in it is gone. This is why Part 3, Task E tells you to ask whether a migration can delete data before you run it.

Part 5 - Running the business day to day

Most days you will not touch code at all. You will use the admin dashboard, which is a normal website with buttons.

Your daily loop, in the admin dashboard

Page	What you do there
Dashboard	Revenue, bookings, views, event and vendor counts at a glance
Review queue	Vendors submit experiences here. You approve or reject them
Bookings	Every booking across every vendor. Search, cancel, check in
Vendors	Create a vendor, set their status and commission rate
Users	Everyone who has signed up. Searchable
Admins	Give or remove staff access

Approving an experience

- 1 Open **Review queue**. Anything waiting is listed.
- 2 Click the experience name to open its full details, photos and ticket prices.
- 3 Click **Approve** to publish it, or **Reject** with a reason.
- 4 A rejection reason is sent back to the vendor, so make it specific and useful.

Adding a vendor

Go to **Vendors**, fill in the business name, contact email and the owner's login email, then click **Create vendor**.

You will be shown a **temporary password** once. Copy it and send it to the vendor by a safe channel. It is never shown again. They change it themselves under Settings after signing in.

Giving a colleague admin access

STOP - They must sign up first

The Admins page only promotes an **existing** account. Ask your colleague to sign up on the public site first, then enter that same email on the Admins page. If you skip this you will see 'No account found for that email' - that is the system working correctly, not a bug.

Commission

Each vendor has a commission rate, set on the Vendors page as a percentage. It is used to calculate your platform revenue on the dashboard. Changing it affects future calculations.

Part 6 - Rules that keep you safe

Short list. Worth reading twice.

Never do these

- **Never paste a secret key into a chat, email or message.** Not to Claude Code, not to anyone. Keys belong only in the .env.local files on your computer and in Vercel's settings screen.
- **Never edit a migration file that has already been run.** It is the record of what your live database looks like. Ask for a new one instead.
- **Never publish a change you have not looked at.**
- **Never make database changes and code changes live at the same moment.** Database first, always.
- **Never share the secret key with a vendor,** however senior. It bypasses every security rule in the system.

Always do these

- **Look at every change locally before publishing.**
- **Ask Claude Code to run the build** before you publish.
- **Keep the documentation true.** When you change how something works, ask: *update the docs to match this change.*
- **Rotate your keys if one is ever shown on screen** - in a screenshot, a screen share, or an error message.

If a key is ever exposed

- 1 In Supabase, go to **Settings, API Keys** and create a **new** secret key.
- 2 Put the new key into both Vercel projects (vendor and admin) and redeploy them.
- 3 Test that creating a vendor still works.
- 4 Only then delete the old key.

Why that order

If you delete the old key first and something is wrong with the new one, you have two broken keys instead of one working one.

Who to trust with what

Person	Should have
You, the owner	Everything: all four services, admin access
Your staff	Admin dashboard access only
A vendor	Their own vendor dashboard login. Nothing else
A developer you hire	GitHub, and their own Supabase and Vercel invitations - never your personal password

Part 7 - When something goes wrong

Work down this list. Most problems are in the first three rows.

What you see	What it usually means	What to do
'command not found' in the Terminal	A tool did not install, or you are in the wrong folder	Re-run Part 1 Step 2. Check you typed cd into the project folder
The local site will not start	Missing or wrong .env.local file	Ask: check my .env.local files are correct and tell me what is missing
'Couldn't create the account' when adding a vendor	The secret key in Vercel is wrong, missing, or has a stray space	Re-copy it from Supabase with the copy button. Redeploy
'No account found for that email'	Working as designed - that person has not signed up yet	Ask them to sign up on the public site first
A dashboard shows all zeros	Usually a data-scoping bug	Ask: the dashboard shows zeros but there is real data. Investigate
The live site did not update	The build failed, or it is still building	Check Vercel for a red build. Ask Claude Code to read the error
You are signed out unexpectedly	Normal session expiry	Sign in again

The universal fallback

If you do not recognise the problem at all, describe it exactly as you experienced it. Precision beats vocabulary.

TYPE THIS INTO CLAUDE CODE

Something is wrong. Here is exactly what I did:
[what you clicked, in order]
Here is what I expected: [what you wanted]
Here is what happened instead: [what you saw]
Investigate and explain before changing anything.

Getting help from a human

If you ever hire a developer, this gets them productive in an afternoon rather than a week:

TYPE THIS INTO CLAUDE CODE

Read CLAUDE.md and everything in docs/, then write me a summary I can send to a developer I am about to hire.

Part 8 - Plain-English dictionary

Word	What it actually means
Repository (repo)	The folder holding your code and its full history
Commit	A saved snapshot of changes, with a label explaining them
Push	Upload your commits, which triggers the live site to update
Main	The official version of your code. What is live comes from here
Deploy / build	Vercel turning your code into the working website
Local / localhost	Running on your own computer only. Nobody else can see it
Terminal	The window where you type commands instead of clicking
Migration	A file of database instructions. Run by hand in Supabase, once, in order
Database	Where all real data lives: bookings, customers, events
RLS (Row Level Security)	The rules deciding who can see which data. Enforced by the database itself, not the website, which is why it is trustworthy
Environment variable	A setting kept outside the code, such as a password. Lives in .env.local and in Vercel
Secret key	A master password that bypasses all security rules. Server only. Never share it
Publishable key	A key that is safe in a browser, because the security rules still apply
Frontend	The part customers see and click
Backend	The part that stores data and enforces rules
API	How two pieces of software talk to each other
Component	A reusable piece of the interface, such as a button
Build error	The code did not compile. The site keeps running on the last good version, so this is safe

Quick reference

Print this page and keep it near your desk.

Starting work

TYPE THIS INTO THE TERMINAL

```
cd ~/Documents/tbdx  
claude
```

The five prompts you will use most

1. STARTING A SESSION

Give me a short summary of the current state of the project and anything that needs my attention.

2. UNDERSTANDING SOMETHING

Explain how [X] works in plain English. Do not change anything.

3. MAKING A CHANGE

[Describe your change]. Then show me the result in the browser before we publish.

4. PUBLISHING

This looks good. Commit it with a clear message and push it to main.

5. UNDOING A MISTAKE

The last change caused a problem. Undo it and put the site back to how it was, then explain what went wrong.

The three golden rules

- 1 Look before you publish.** Always view the change locally first.
- 2 Database first, code second.** Run the migration in Supabase before publishing the code that needs it.
- 3 Keys never go in a chat.** Only in .env.local and Vercel settings.

Where things live

Need to...	Go to
Approve an experience	admin-tbdx.vercel.app, Review queue
See all bookings	admin-tbdx.vercel.app, Bookings
Add a vendor	admin-tbdx.vercel.app, Vendors
Look at the data directly	supabase.com, Table Editor
Run a migration	supabase.com, SQL Editor
Check if a deploy worked	vercel.com, your project, Deployments
Change a secret key	vercel.com, Settings, Environment Variables

This handbook describes the project as handed over. As you change the project, ask Claude Code to keep docs/ up to date - the technical reference it reads every session. This PDF is the human-facing companion to those documents.